DETAILED PROJECT REPORT

Robust Bank Account Management System

A C-Language Implementation

Foundations of Programming — Mini-Project

Instructor: Nikhil Soniminde

Team Members

Name	PRN
Abhanshi Dubey	1262253325
Sharvari Harbude	1262250522
Ishan Khatti	1262251872
Adit Kothari	1262251552
Soham Kopurwar	1262250813

1. Introduction

This report provides a comprehensive technical analysis of the Robust Bank Account Management System, a mini-project developed as part of the Foundations of Programming course. The system is implemented entirely in the C programming language and simulates essential banking operations through a console-based interface.

The project serves a dual purpose: it functions as a practical banking simulation while simultaneously demonstrating the power and versatility of fundamental C programming constructs. It illustrates how academic programming principles can be effectively applied to build functional, real-world-inspired systems.

1.1 Motivation

Manual tracking of financial transactions is inherently error-prone and inefficient. This project addresses that problem by creating a digital simulation of core banking functions, allowing users to interact with their account data through a structured and validated interface.

1.2 Scope

The system is scoped as an in-memory, session-based banking simulation. It does not persist data between runs (though this is identified as a future enhancement), but it provides complete transactional functionality within a single program session.

2. Project Objectives

The project was designed with the following key objectives:

- Create a simple yet functional banking simulation system in C.
- Implement and demonstrate core C programming concepts including data structures, control flow, and input/output.
- Develop logical and structured programming skills through hands-on application.
- Build a user-friendly, menu-driven command-line interface for seamless interaction.
- Validate user inputs rigorously to prevent invalid or harmful operations such as overdrafts.
- Design modular and readable code that adheres to structured programming principles.

3. Features of the System

The Bank Account Management System provides six core features, each serving a distinct banking operation:

#	Feature	Description
1	Display Account Balance	Shows the current account balance reflecting all deposits and withdrawals. Demonstrates dynamic state management.
2	Process Fund Deposits	Allows users to add funds. Input validation ensures only positive amounts are accepted before updating the balance.
3	Track Cumulative Deposits	Records the total sum of all funds ever deposited using an accumulator variable, separate from current balance.
4	Manage Fund Withdrawals	Facilitates fund removal with critical overdraft prevention checks ensuring withdrawal does not exceed balance.
5	Monitor Cumulative Withdrawals	Displays the aggregate amount of all withdrawals made during the session for transaction overview.
6	Program Termination	Provides a clean, controlled exit mechanism from the application loop, terminating the user session gracefully.

4. Core C Programming Concepts Used

This project is a practical showcase of fundamental C programming constructs. Below is a detailed breakdown of each concept employed:

4.1 Data Types and Variables

The program uses int and float data types to store account balance, deposit amounts, and withdrawal values. Proper variable declaration ensures efficient memory allocation and type-safe data handling throughout the program.

4.2 Conditional Control Flow — if-else

The if-else construct is used extensively, most critically in the withdrawal feature. Before deducting funds, the program checks:

if (balance < withdraw) → Deny transaction (Insufficient Funds)

else → Proceed with deduction

This prevents overdrafts and maintains data integrity.

4.3 Multi-way Branching — switch-case

The main menu is implemented using a switch-case statement, providing a clean and efficient way to branch into one of six operations based on user input. This is preferred over nested if-else chains for menu-driven programs as it improves clarity and performance.

4.4 Iterative Constructs — do-while Loop

The application's main execution is wrapped in a do-while loop, which guarantees at least one presentation of the menu before checking the exit condition. The loop continues running until the user selects Option 6 (Exit), allowing continuous transactions in a single session.

4.5 Arithmetic Operations

Basic mathematical operators are applied directly to the balance variable:

- Deposit: balance += deposit_amount
- Withdrawal: balance -= withdrawal_amount
- Cumulative tracking uses additional accumulator variables updated with += on each transaction.

4.6 Standard Input — scanf()

User inputs for menu selection and transaction amounts are captured using scanf(). This function reads and parses formatted data from standard input, enabling interactive and dynamic program execution based on real-time user responses.

5. Program Execution Flow

The program follows a clearly defined, sequential execution flow:

Step	Action	Details
1	Initialization	Program starts and prompts the user to enter their initial account balance.
2	Menu Display	The main menu is presented with all 6 available banking options.
3	User Selection	The user inputs a choice, which is read and validated via switch-case.
4	Operation Execution	The selected operation (deposit, withdraw, balance check, etc.) is executed and account state is updated.
5	Loop Continuation	Control returns to the menu display, allowing further transactions.
6	Exit	If the user selects Option 6, the do-while loop terminates and the program ends cleanly.

Execution Flow Diagram (Logical)

START → Enter Initial Balance → Display Menu → User Selects Option

↓

Option 1: Show Balance | Option 2: Deposit | Option 3: Show Total Deposits

Option 4: Withdraw (with validation) | Option 5: Show Total Withdrawals | Option 6: Exit

↓

Loop back to Menu → Until Option 6 → END

6. Core Transaction Logic

6.1 Deposit Mechanism

When a deposit is initiated, the program accepts a positive amount from the user and executes:

```
balance += deposit;
```

Simultaneously, a cumulative deposit tracker is updated:

```
total_deposits += deposit;
```

This ensures both the current balance and the historical deposit record are maintained accurately.

6.2 Withdrawal Mechanism

The withdrawal process includes a critical pre-validation check:

```
if (balance < withdraw) → Print error: Insufficient funds
else → balance -= withdraw; total_withdrawals += withdraw;
```

This two-step mechanism (validate → execute) ensures financial integrity and prevents account overdrafts under any circumstance.

6.3 Balance Inquiry

A simple output statement prints the current value of the balance variable, reflecting all completed deposits and withdrawals up to that point in the session.

6.4 Cumulative Tracking

Two separate accumulator variables independently track the sum of all deposits and withdrawals respectively. These are never reset within the session, providing a complete transaction history view.

7. Conclusion

The Robust Bank Account Management System successfully demonstrates how foundational C programming concepts can be integrated to create a functional, user-friendly application. The project achieves all its stated objectives:

- A fully working console-based banking simulation was implemented.
- Core C concepts — control flow, loops, conditionals, arithmetic, and I/O — are effectively demonstrated.
- The system is logically structured, readable, and maintainable.
- Input validation ensures data integrity and prevents erroneous operations.
- The menu-driven interface provides an intuitive user experience.

The project serves as an excellent foundation for understanding how real-world software systems are structured, and how even complex-seeming applications can be decomposed into simple, manageable logic blocks.

8. Future Enhancements

The team has identified several areas for future development to increase the system's capability and robustness:

Enhancement	Description
Enhanced User & Account Management	Implement unique account identifiers and comprehensive user profiles, including personal data and full transaction history.
Secure Authentication Mechanisms	Introduce hashed password storage and multi-factor authentication to safeguard user accounts from unauthorized access.
Persistent Data Storage	Integrate file I/O operations or database connectivity (e.g., SQLite) to retain data across application sessions.
Scalable Multi-Account Framework	Develop architecture using arrays/linked lists to support and manage multiple user accounts simultaneously.
GUI Interface	Transition from a console-based UI to a graphical user interface using libraries such as GTK or Qt.
Transaction History Log	Implement a log file or in-memory list that records every transaction with timestamps for auditability.

9. Team & Acknowledgements

This project was developed as a collaborative team effort under the guidance of Instructor Nikhil Soniminde, as part of the Foundations of Programming course.

Team Member	PRN	Role
Abhanshi Dubey	1262253325	Core Logic & Implementation
Sharvari Harbude	1262250522	Testing & Validation
Ishan Khatti	1262251872	Documentation & Presentation
Adit Kothari	1262251552	UI Flow & Menu Design
Soham Kopurwar	1262250813	Code Review & Debugging

The team acknowledges the guidance and mentorship of Instructor Nikhil Soniminde, whose structured approach to teaching programming fundamentals made this project possible.